

Análisis Individual Caso B C02: Plataforma de inscripción escolar

Fundamentos de ingeniería de software

U1-E02 Proyecto (Re:Biblioteca)

Alexis Rubio Galvan



30/09/2026

Presentación del caso y consecuencias

La plataforma de inscripción escolar fue diseñada para consultar materias, seleccionar grupos y generar un comprobante de inscripción. Durante las pruebas iniciales, realizadas con pocos usuarios, las funciones principales respondían correctamente; sin embargo, al llegar el periodo de mayor demanda aparecieron problemas que muestran una preparación insuficiente para las condiciones reales de operación. La lentitud del sistema, el procesamiento duplicado de solicitudes y la generación de comprobantes después de agotarse los cupos afectan directamente la confiabilidad del proceso de inscripción.

Para los estudiantes, la principal consecuencia es la incertidumbre sobre si su inscripción realmente quedó registrada y si el comprobante que recibieron representa un lugar válido. Para el personal administrativo, los errores generan trabajo adicional al tener que revisar inscripciones, cupos y comprobantes. El equipo de soporte recibe reportes, pero al no clasificarlos por prioridad ni asociarlos con una versión específica, pierde tiempo tratando de determinar cuáles incidentes son más urgentes y qué cambio pudo originarlos. Finalmente, para el equipo de desarrollo, los cambios de requisitos que llegan por distintos canales y la ausencia de un plan de reversión dificultan mantener una versión estable durante el periodo crítico.

En conjunto, el caso muestra que el problema no se limita a una falla técnica aislada. Existen fallas de proceso que se vuelven más graves cuando aumenta la demanda: se probaron pocas condiciones de carga, no se controló adecuadamente la concurrencia, no existe una fuente única para las decisiones de requisitos, falta un procedimiento documentado para regresar a una versión estable y soporte carece de un esquema formal de priorización y trazabilidad.

Cinco problemas identificados

● Problema 1: Pruebas de carga insuficientes

Causa: Las pruebas iniciales se realizaron con pocos usuarios y no reprodujeron las condiciones del periodo de mayor demanda.

Consecuencia: La plataforma presenta lentitud cuando aumenta el número de usuarios, lo que puede retrasar operaciones críticas como consultar grupos, seleccionar materias y confirmar la inscripción.

Práctica de Ingeniería de Software: Realizar pruebas de rendimiento, carga y escalabilidad antes del periodo de inscripción, utilizando escenarios que representen el número esperado de usuarios simultáneos y también condiciones de mayor exigencia.

Evidencia del caso: Durante las pruebas con pocos usuarios las funciones principales respondían correctamente, pero el día de mayor demanda la plataforma se vuelve lenta.

● Problema 2: Falta de control de concurrencia en la asignación de cupos

Causa: Las solicitudes que llegan al mismo tiempo no están siendo coordinadas de manera que la consulta de disponibilidad y la asignación del cupo se comporten como una operación

consistente.

Consecuencia: Algunas solicitudes se procesan dos veces y ciertos estudiantes reciben un comprobante aunque el cupo ya se había agotado. Esto puede producir registros inconsistentes y conflictos administrativos.

Práctica de Ingeniería de Software: Implementar transacciones y mecanismos adecuados de control de concurrencia para que la comprobación y actualización del cupo se realicen de forma consistente.

Evidencia del caso: El caso indica que algunas solicitudes se procesan dos veces y que determinados estudiantes reciben un comprobante aunque el cupo de la materia ya se agotó.

● **Problema 3: Cambios de requisitos sin una fuente única de decisiones**

Causa: Los cambios de requisitos llegan por correo electrónico y mensajes separados, sin una lista centralizada que indique cuál decisión está vigente.

Consecuencia: El equipo puede implementar información desactualizada, duplicar trabajo o interpretar de manera diferente un mismo cambio, especialmente cuando se acerca el periodo de mayor demanda.

Práctica de Ingeniería de Software: Utilizar un mecanismo formal de gestión de requisitos y cambios, donde cada solicitud tenga descripción, prioridad, responsable, revisión y estado.

Evidencia del caso: Los cambios de requisitos llegan por correo y mensajes separados y no existe una lista única de decisiones.

● **Problema 4: Ausencia de control de versiones y plan de reversión**

Causa: No existe un plan documentado para regresar a una versión estable cuando una modificación provoca una falla.

Consecuencia: Si un cambio introdujera un error durante el periodo de inscripción, el equipo podría tardar más en recuperar una versión funcional, aumentando el tiempo de afectación para estudiantes y personal.

Práctica de Ingeniería de Software: Utilizar control de versiones, ramas de desarrollo y un procedimiento documentado de reversión. Esto permite recuperar versiones anteriores y controlar los cambios realizados al sistema.

Evidencia del caso: El caso establece que no existe un plan documentado para regresar a una versión estable.

● **Problema 5: Gestión deficiente de incidencias y falta de trazabilidad**

Causa: Soporte recibe reportes, pero no los clasifica por prioridad ni puede relacionarlos con una versión específica.

Consecuencia: Los incidentes críticos pueden no distinguirse rápidamente de los de menor impacto y resulta más difícil identificar patrones, relacionar una falla con un cambio y coordinar la respuesta.

Práctica de Ingeniería de Software: Establecer un sistema de gestión de incidencias con identificador único, descripción, categoría, prioridad, responsable, estado y versión afectada.

Evidencia del caso: Soporte recibe reportes, pero no los clasifica por prioridad ni puede relacionarlos con una versión específica.

Tres acciones prioritarias

- **Acción 1: Ejecutar pruebas de carga y concurrencia antes del siguiente periodo crítico**

Orden: 1 (Máxima prioridad).

Justificación: El problema más visible aparece cuando la demanda aumenta y coincide con errores de procesamiento. Antes de modificar ampliamente el sistema conviene reproducir el comportamiento bajo condiciones controladas para conocer el límite de la plataforma y localizar los cuellos de botella.

Costo o limitación: Requiere tiempo de preparación, herramientas para simular usuarios y, posiblemente, infraestructura adicional para ejecutar pruebas sin afectar el servicio real.

Cómo comprobar la mejora: Ejecutar escenarios con diferentes cantidades de usuarios simultáneos y registrar tiempo de respuesta, porcentaje de errores, solicitudes duplicadas y capacidad de procesamiento. Comparar estos resultados con una línea base antes de los cambios.

- **Acción 2: Corregir el proceso transaccional de asignación de cupos**

Orden: 2 (Prioridad alta).

Justificación: Evita que dos solicitudes concurrentes consuman o confirmen el mismo cupo y protege directamente la integridad del proceso de inscripción.

Costo o limitación: Puede requerir modificaciones en la lógica del servidor y en las operaciones de base de datos; además, las pruebas deben contemplar escenarios simultáneos y casos límite.

Cómo comprobar la mejora: Simular a varios estudiantes intentando obtener simultáneamente el último cupo disponible y verificar que el sistema asigne correctamente el lugar, rechace las solicitudes excedentes y no genere comprobantes inválidos.

- **Acción 3: Centralizar cambios, incidencias y versiones**

Orden: 3 (Prioridad preventiva).

Justificación: Permite que desarrollo, soporte y las áreas solicitantes trabajen con la misma información. También facilita identificar qué cambio corresponde a cada versión y qué

incidentes requieren atención inmediata.

Costo o limitación: Implica seleccionar y configurar una herramienta o repositorio, definir responsables y dedicar tiempo a que todas las áreas adopten el procedimiento. Al principio puede aumentar el trabajo administrativo.

Cómo comprobar la mejora: Revisar una muestra de cambios e incidentes y comprobar que cada registro tenga responsable, prioridad, estado, fecha, versión relacionada y evidencia de resolución. Además, verificar que exista una única fuente vigente para las decisiones de requisitos.

Reflexión personal

Este caso me permite entender que un sistema no puede considerarse confiable solamente porque sus funciones principales trabajen correctamente durante una prueba sencilla. En una plataforma de inscripción, el comportamiento cambia cuando muchos estudiantes realizan operaciones al mismo tiempo y cuando existen cupos limitados que deben mantenerse consistentes. También resulta evidente que la ingeniería de software incluye procesos que van más allá de programar: probar bajo condiciones reales, controlar cambios, mantener versiones recuperables y registrar los incidentes de forma ordenada.

A partir de este análisis, considero importante acostumbrarme a pensar en el comportamiento del sistema antes, durante y después de una liberación. Una función puede parecer correcta de manera individual y aun así fallar cuando interactúa con muchos usuarios, cambios simultáneos o información desactualizada. Por ello, una de las principales enseñanzas del caso es que la calidad depende tanto de la solución técnica como de la disciplina con la que el equipo gestiona requisitos, pruebas, versiones e incidencias.

Fuentes consultadas y declaración de uso de IA

- Fuentes:

- IBM. (s.f.). ¿Qué son las pruebas de rendimiento? IBM Think.

<https://www.ibm.com/mx-es/think/topics/performance-testing>

- IBM. (s.f.). ¿Qué es la gestión de requisitos? IBM Think.

<https://www.ibm.com/mx-es/think/topics/what-is-requirements-management>

- PostgreSQL Global Development Group. (s.f.). Transactions. PostgreSQL Documentation.

<https://www.postgresql.org/docs/current/tutorial-transactions.html>

- Git. (s.f.). Acerca del Control de Versiones. Pro Git.

<https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>

- Atlassian. (s.f.). Gestión de incidentes: procesos, prácticas recomendadas y herramientas.

<https://www.atlassian.com/es/incident-management>

- Declaración: Para la redacción y corrección de este análisis empleé una herramienta de Inteligencia Artificial como apoyo puntual para organizar y mejorar la redacción. El análisis del caso, la selección de los problemas, las acciones propuestas y la revisión final corresponden al trabajo del alumno.